

✓ Stage 3 — Data Extraction

Objective

The objective of this stage is to extract the specific information required for subsequent data validation, aggregation, analysis, and visualization.

The transaction timestamp is converted into a proper datetime format so that useful temporal attributes such as date, month, and year can be extracted.

A focused dataframe containing the attributes required for downstream analysis is then created from the original transaction dataset.

Operations Performed

1. Loaded the transaction dataset.
2. Converted the timestamp column into datetime format.
3. Extracted date, month, and year from the timestamp.
4. Selected the attributes required for subsequent analysis.
5. Verified the dimensions of the extracted dataset.
6. Displayed the extracted attributes.
7. Inspected sample extracted transactions.

Output

The result of this stage is a focused dataframe containing the transaction, temporal, amount, status, fraud, geographical, device, network, and time-related attributes required for the following stages.

```
# =====
# STAGE 3 – CONTAINER 1
# SOURCE DATASET
# =====

import pandas as pd
import numpy as np

df = pd.read_csv("upi_transactions_2024.csv")

print("SOURCE DATASET")
print("=" * 60)

print(f"Rows      : {df.shape[0]:,}")
print(f"Columns   : {df.shape[1]}")
```

```
SOURCE DATASET
=====
Rows      : 250,000
Columns   : 17
```

```
# =====
# STAGE 3 – CONTAINER 2
# TIMESTAMP AND TEMPORAL EXTRACTION
# =====

df["timestamp"] = pd.to_datetime(
```

```

df["timestamp"],
errors="coerce"
)

df["date"] = df["timestamp"].dt.date
df["month"] = df["timestamp"].dt.month
df["year"] = df["timestamp"].dt.year

print("Temporal extraction completed.")
print("=" * 60)

print("Original temporal field : timestamp")
print("Extracted fields      : date, month, year")

print("\nSample temporal information:")
display(
    df[
        ["timestamp", "date", "month", "year"]
    ].head()
)

```

Temporal extraction completed.

=====

Original temporal field : timestamp

Extracted fields : date, month, year

Sample temporal information:

	timestamp	date	month	year	
0	2024-10-08 15:17:28	2024-10-08	10	2024	
1	2024-04-11 06:56:00	2024-04-11	4	2024	
2	2024-04-02 13:27:18	2024-04-02	4	2024	
3	2024-01-07 10:09:17	2024-01-07	1	2024	
4	2024-01-23 19:04:23	2024-01-23	1	2024	

```

# =====
# STAGE 3 – CONTAINER 3
# REQUIRED ATTRIBUTE EXTRACTION
# =====

```

```

required_columns = [
    "transaction id",
    "timestamp",
    "date",
    "month",
    "year",
    "transaction type",
    "merchant_category",
    "amount (INR)",
    "transaction_status",
    "fraud_flag",
    "sender_state",
    "device_type",
    "network_type",
    "hour_of_day",
    "day_of_week",
    "is_weekend"
]

extracted_df = df[required_columns].copy()

```

```
print("REQUIRED ATTRIBUTES EXTRACTED")
print("=" * 60)

print(f"Extracted rows    : {extracted_df.shape[0]:,}")
print(f"Extracted columns  : {extracted_df.shape[1]:,}")
```

```
REQUIRED ATTRIBUTES EXTRACTED
=====
Extracted rows    : 250,000
Extracted columns : 16
```

```
# =====
# STAGE 3 – CONTAINER 4
# EXTRACTED DATASET PROFILE
# =====

extracted_profile = pd.DataFrame({
    "Column": extracted_df.columns,
    "Data Type": extracted_df.dtypes.astype(str).values,
    "Non-Null Values": extracted_df.notna().sum().values,
    "Unique Values": extracted_df.nunique(dropna=True).values
})

display(extracted_profile)
```

	Column	Data Type	Non-Null Values	Unique Values	
0	transaction id	object	250000	250000	
1	timestamp	datetime64[ns]	250000	248610	
2	date	object	250000	365	
3	month	int32	250000	12	
4	year	int32	250000	1	
5	transaction type	object	250000	4	
6	merchant_category	object	250000	10	
7	amount (INR)	int64	250000	10355	
8	transaction_status	object	250000	2	
9	fraud_flag	int64	250000	2	
10	sender_state	object	250000	10	
11	device_type	object	250000	3	
12	network_type	object	250000	4	
13	hour_of_day	int64	250000	24	
14	day_of_week	object	250000	7	
15	is_weekend	int64	250000	2	

Next steps: [Generate code with extracted_profile](#) [New interactive sheet](#)

```
# =====
# STAGE 3 – CONTAINER 4
# EXTRACTED DATASET PROFILE
# =====
```

```
extracted_profile = pd.DataFrame({
    "Column": extracted_df.columns,
    "Data Type": extracted_df.dtypes.astype(str).values,
    "Non-Null Values": extracted_df.notna().sum().values,
    "Unique Values": extracted_df.nunique(dropna=True).values
})

display(extracted_profile)
```

	Column	Data Type	Non-Null Values	Unique Values	
0	transaction id	object	250000	250000	
1	timestamp	datetime64[ns]	250000	248610	
2	date	object	250000	365	
3	month	int32	250000	12	
4	year	int32	250000	1	
5	transaction type	object	250000	4	
6	merchant_category	object	250000	10	
7	amount (INR)	int64	250000	10355	
8	transaction_status	object	250000	2	
9	fraud_flag	int64	250000	2	
10	sender_state	object	250000	10	
11	device_type	object	250000	3	
12	network_type	object	250000	4	
13	hour_of_day	int64	250000	24	
14	day_of_week	object	250000	7	
15	is_weekend	int64	250000	2	

Next steps: [Generate code with extracted_profile](#) [New interactive sheet](#)

```
# =====
# STAGE 3 – CONTAINER 5
# SAMPLE EXTRACTED TRANSACTIONS
# =====

print("SAMPLE EXTRACTED TRANSACTIONS")
print("=" * 60)

display(
    extracted_df.head(3)
)
```

SAMPLE EXTRACTED TRANSACTIONS

=====

	transaction_id	timestamp	date	month	year	transaction_type	merchant_category	amount (INR)	transaction_status
0	TXN0000000001	2024-10-08 15:17:28	2024-10-08	10	2024	P2P	Entertainment	868	SUCCESS
1	TXN0000000002	2024-04-11 06:56:00	2024-04-11	4	2024	P2M	Grocery	1011	SUCCESS
2	TXN0000000003	2024-04-02 13:27:18	2024-04-02	4	2024	P2P	Grocery	477	SUCCESS

Stage 3 — Extraction Summary

The transaction timestamp was successfully converted into datetime format and used to derive date, month, and year attributes.

A focused dataframe was created containing the transaction, temporal, amount, status, fraud, geographical, device, network, and time-related attributes required for subsequent processing.

The extracted dataframe retains the complete set of transaction records while providing a more focused collection of attributes for downstream operations.

The output of this stage will be used in Stage 4 for data validation and cleaning. The derived temporal fields and selected attributes will be checked for consistency, valid formats, unexpected values, and other data-quality issues before analysis begins.